NewBear Computing Store Ltd

HEAD OFFICE: 40 BARTHOLOMEW ST. NEWBURY BERKS. RG14 5LL Tel: (0638) 30505 Telex: 848507 NCS

Newbear Computing Store

77 - 68 Disc Controller Board

Design Note 82

Contents

77 - 78 Disk Controller Board

Users Manual

1. Introduction

The 77-78 Disk Controller board provides an interface between the 77-78 microcomputer system and between one and four full size floppy disk drives. It has a very simple yet powerful command set which allows full control of the disk drives, and includes comprehensive error detection both in the commands and in the data written to or read from the disk.

Disk Format

This controller handles disks which are hard sectored; that is each sector is indicated by a hole on the disk itself, with a special hole to indicate the start of the track.

There are 77 tracks on each disk, each track being divided into 16 sectors of 256 bytes each. This gives a total disk capacity of 315392 bytes, more than 20% greater than the standard IBM format.

2. Hardware Description

The disk is based around the 6800 microprocessor, with the data interface to the disk being via the 6852 SSDA (Synchronous Serial Data Adaptor). The circuit will be described in sections, based on the block diagram, figure 2.1

(a) Address Buffers

The address buffers allow the 77-78 system to access both the RAM and the status latch on the disk controller board. Address lines A15-A10 are decoded to provide a board select line, placing the disk controller board between 8400H and 87FFH. The RAM occupies 8400H to 87FEH, with the status latch at 87FFH. Since both the 77-68 system and the on-board 6800 require access to the RAM,the address buffers on lines A9-A0 are only enabled while the controller is inactive, and are disabled during operation. While the controller is running, gates IC13, IC1 (1,2) and IC12 (1,2,4,5,6) allow the status latch to be accessed.

(b) Control Interface

The control interface generates all the control signals for enabling and disabling the 77-78 buffers, and provides the strobes for accessing the RAM from the 77-68. In addition, it provides the strobe which causes the controller board to commence its operation.

Buffer enabling is all controlled by the BUS ENABLE (BA) signal from pin 7 of the 6800 When the controller is inactive, BA is high which enables the address buffers and enables the strobes to the data buffers via IC15 (8,9,10 and 11,12,13).

Also, the control interface will enable the status latch output onto bit 7 of the data bus via IC2 (17,18), when the processor reads from address 87FFH. The status latch may be read regardless of the state of the controller.

The controller board operation is started by a write to address 87FFH. This causes a pulse to be generated by IC34 which feeds the interrupt (IRQ) imput of the 6800. This commences operation as described in section 3 (software description).

(c) Data Buffer

These are bi-directional buffers which allow the 77-68 to access the RAM on the controller board. They are enabled by the control interface, but, as with the address buffers, only when the controller is inactive.

(d) M6800 Microprocessor

The 6800 microprocessor is the heart of the disk controller board. It controls all disk drives, such as moving the read/write head to the appropriate track and loading the head onto the disk, and also handles all data transfers to and from the disk.

When the disk-controller is inactive, the 6800 is in an idle condition whereby all its bus lines (address, data, control) are in a tri-state condition so that the 77-68 system may access the RAM. When an interrupt occurs, the 6800 exits the idle condition and executes the command requested by the 77-68 system. When the operation is complete, the 6800 returns to the idle condition.

(e) Program Memory

The problem memory is a single 2708 UV erasable PROM which holds all of the program executed by the 6800 processor. The code occupies approximately 900 bytes of the 1024 bytes available. The program is described in section 3.

(f) Scratchpad Memory

This is the RAM referred to above. It consists of 2 x 2114 (or TMS 4045) 1K x 4 bit RAMS to provide a full 1K x 8 bits of RAM. This RAM has two purposes. Firstly it holds all the data required by the disk controller board for its operation, and secondly it holds the two buffers which are accessed by the 77-68, the read and write buffers. Each of these buffers is 256 bytes long, the same as a sector on the disk.

g) Clock Generator

The clock generator is dual purpose, providing a clock for both the 6800 and for the serial data interface. The clock to the 6800 is a two phase 1MHz 50-50 mark-space square wave. The serial interface requires two clocks, one a 1MHz 50-50 mark-space square wave, and the other a more complex wave form which is used to produce the write data wave form to the disk drive. All of the clock wave forms are derived from a 10MHz crystal oscillator to ensure stability.

i) Serial Data Interface

The major part of this is the 6852 SSDA which converts parallel data from the 6800 to serial, and serial data from the disk into parallel.

The recovered clock and data signals from the disk drive are fed to the SSDA via monostables which are used to clean the wave forms up and to ensure that the clocking edge falls within the data pulse.

j) Disk Drive Control and Status Interface

This is a set of output and imput parts which control the drive functions such as drive select, loading the read/write heåd etc., and monitor the drive status lines such as track 0, write protect, sector etc. All of the signals between the controller and the disk drive are shown in figure 2.2.

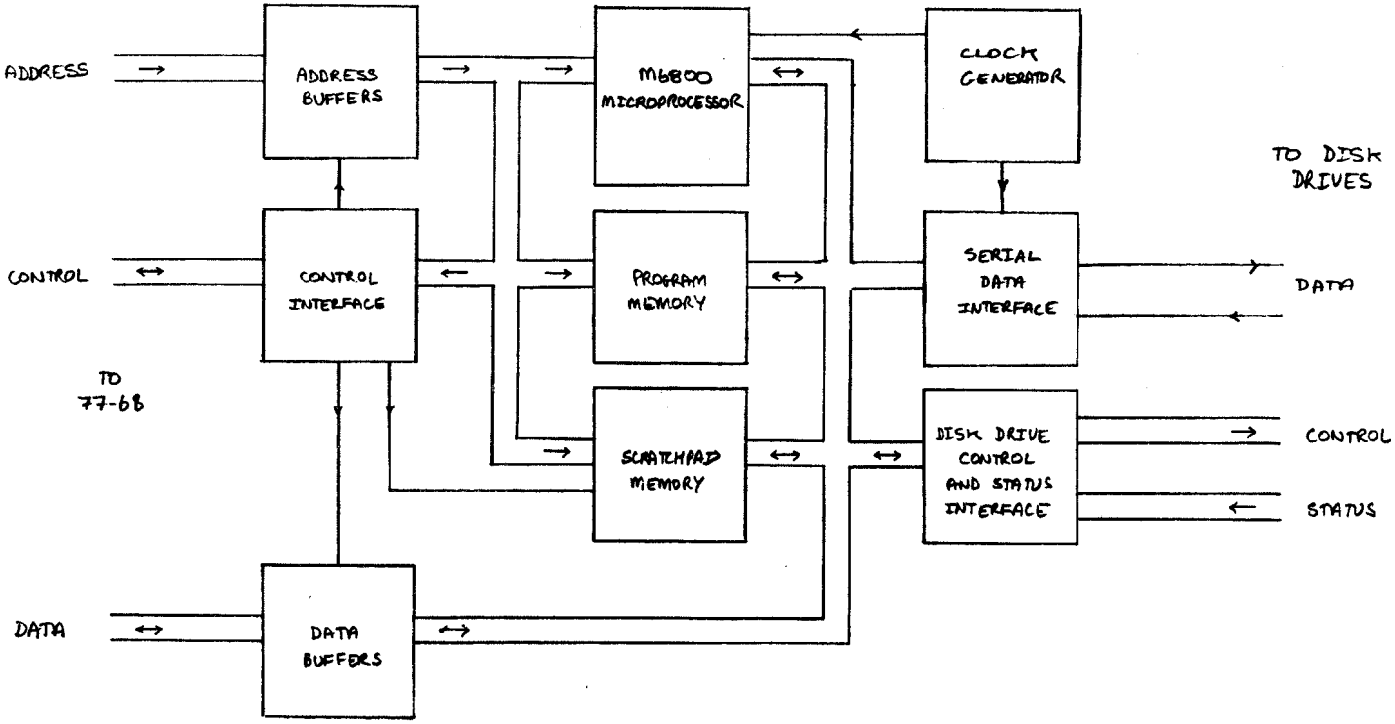


Figure 2.1 77-68 DISK CONTROLLER BLOCK DIAGRAM

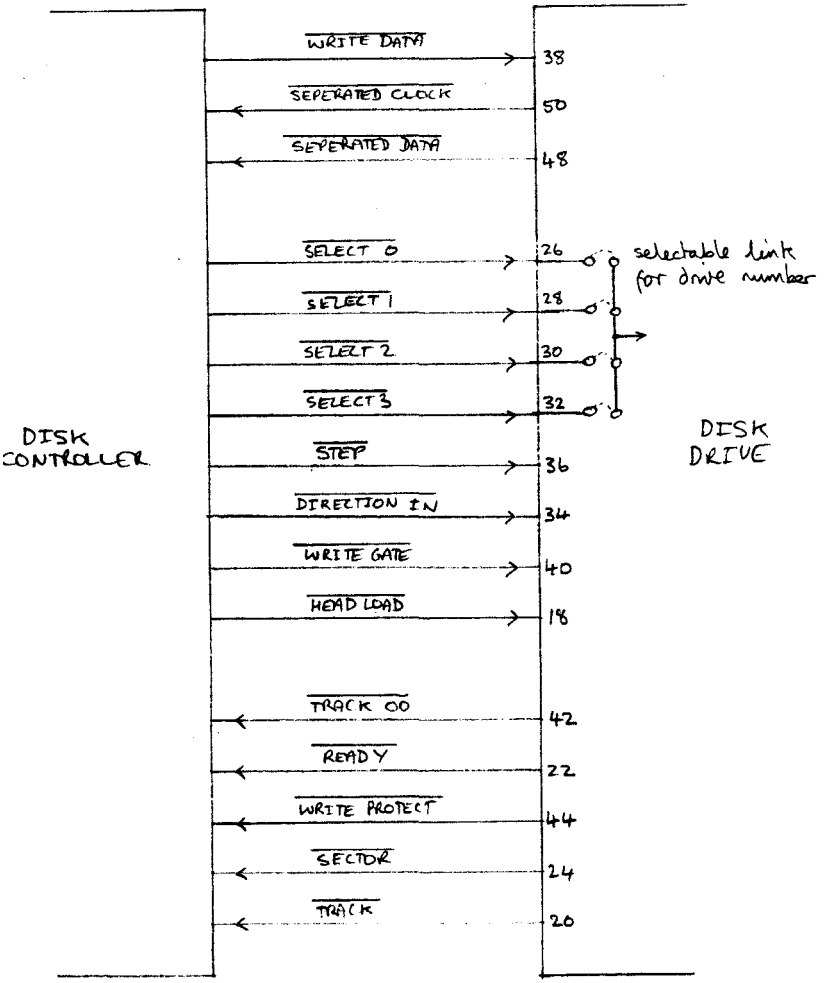


Figure 2.2 Disk Controller to disk drive Hardware Interface

3. Software Description

Disk Sector Format

Each sector on a disk has to contain more than just 256 bytes of data allocated to it. There is also the housekeeping and clocking information.

A complete sector is made up as follows:-

```
bytes    0-15 - all zeros - clock run-in
byte     16   - 10000001  - synchronisation byte
bytes    17-27 - all zeros
byte     28   - TRKNUM    - track number of current sector
byte     29   - SCTNUM    - sector number of current sector
bytes    30-41 - all zeros
bytes    42-297- data      - 256 data bytes
bytes   298-299- checksum  - 16 bit checksum on data
bytes   300-315- all zeros - clock over-run
```

Bytes 0-15 are all zeros, which allow the clock recovery circuitry on the disk drive to become synchronised to the information read from the disk Byte 16 synchronises the 6852 SSDA to the 8 bit data bytes.

Software

All the major routines held in the 2708 program memory on the disk controller are shown in outline flow chart form in figures 3.1. to 3.9. In addition there are a number of minor routines associated with control of the interface or the drive which are adequately described by the comments in the program listing in appendix C.

The major routines are more fully described in the following sections

3.1/ Disk Hardware Interface

Up to four disk drives may be controlled by the controller board. All control and status information passes through a single 8 bit output port and a single 8 bit input port. Both of these are at address 8000H.

Control Port

bit	7	6	5	4	3	2	1	0
	UL	HL	WG	IN	ST	EN	DI	DO

UL = Unload Read/Write Head
HL = Head Load
WG = Write Gate (enable write to disk)
IN = Set head step direction to IN
ST = Step head by one track
EN = Enable drive selection
DI,DO= Drive njmber to select

Status Port

bit	7	6	5	4	3	2	1	0
	HS	TZ	RDY	WP	S3	S2	S1	SO

HS = Head Loaded
TZ = Head at track zero
RDY = Drive in ready
WP = Disk in write protected
S3-SO= Current Disk Sector

3.2 Restrt and Rentry routines (figure 3.1)

The routine RESTRT will be entered whenever the system reset switch is operated. It will initialise the 6852 serial interface chip, and will then set to track zero the read/write heads on all the drives attached to the controller. This has to be done since there is no absolute reading as to the position of the read/write head, only a relative one found by counting track step pulses from track zero. Whenever track zero is requested, the controller executes this special seek looking for bit 6 of the status port to become one.

Once all the drives have been initialised, sector eight of track zero will be brought off the disk in drive zero into the read buffer. This allows a loader program at this track and sector to be brought automatically off the disk to be moved into user RAM, and executed to bring in a full operating system. 'This facility is used by BEARDOS, the disk operating system designed to work with the 77-68 and this disk controller board.

The main command loop is now entered at RENTRY. This is the re-entry point from all the sub routines called by the command loop. The processor will enable interrupts, and then execute a WAI instruction, which causes the 6800 to halt and set its address, data, and control buses to tri-state allowing the 77-68 to access the RAM on the controller. When the controller is to be started up, the 77-68 writes to address 87FFH which generates an interrupt to start the program running again.

The control loop now reads the op code from the RAM, checks for valid range (1-8) loads the address of the relative routine and calls it. On return, an error code in ACC-B is saved in the RAM, and the processor returns to its halted condition.

3.3./ Read Routine (Figures 3,2,3,3,4)

With any mass storage system, errors will occur from time to time. Some of these are non-recoverable (eg data corruption on the disk) whereas some may be recovered by re-reading the sector. The READ routine on the controller does just that. Referring to figure 3.2., the top level READ flowchart, it may be seen that nine reads are executed before an un-recoverable error is reported. If the first three reads are unsuccessful, the head is stepped one track and three more reads are attempted. If these are also unsuccessful, a seek to track zero is executed, and only if the last three reads are unsuccessful, is an error reported. This method allows for dirt on the head, power supply glitches etc. to be ironed out.

The actual disk read routine, READ 1 (figure 3.4.), is called once the read/write head is at the required track. The head is loaded, the required sector is found, and the sector is read into the Read Buffer.

Once in,the following checks are performed on the information:-

(1) The checksum is calculated for the data read, and compared with the checksum recovered from the disk.

(11) The track and sector numbers recovered from the disk are compared with the requested track and sector numbers.

If either of these checks fails, an error will be reported

3/4 WRITE routine (figures 3,5, and 3,6,)

There are two modes of operation of the WRITE routine, depending on whether verifieation was requested or not:-

(1) If verifying was not requested, the sector will be written and the routine will return immediatley.

(11) If verifying was requested, the write will be followed by a READ3 to verify that the sector can be read correctly. If not this procedure will be done twice more before an error is reported, as in the READ routine.

The actual disk write routine, WRITE1 is shown in fig. 3.6. Before writing the sector to the disk, all of the housekeeping bytes must be setup, as mentioned at the beginning of this section, and the data checksum is calculated. Then the head is stepped to the required track, the required sector found and the serial interface enabled for writing. Then the sector is written, and the serial face is disabled.

3.4./ GETTRK routine (figure 3.7.)

This routine will seek the requested track. If track zero was specified the special routine to seek track zero will be called, as this allows the hardware and software to check that it is in step as far as track number is concerned, as a hardware signal is produced when the head is at track zero. Otherwise the head will be moved the required number of steps from the current head position.

3.5./ INIT routine (figure 3.8)

This is the routine which initialises the required drive. It is called whenever RESTRT is executed to initialise all of the drives present, or it may be called as a user command from the 77-68. It initialises the control port (just a precaution), selects the required drive, seeks to track zero, and then de-selects the drive.

3.6./ FORMAT routine (figure 3.9.)

The FORMAT routine is used to initialise the disk itself. It will write to the disk all the track and sector and clock information, and set all the data in all sectors to zero. It first seeks track zero to initialise the drive, then steps out to the last track (track 76). Then every sector is written to in that track. After each track is written, the head will step in to the next track, and so on until track zero has been written, when the routine will exit.

All disks MUST be formatted before use, otherwise it will be impossible to read from the disk.

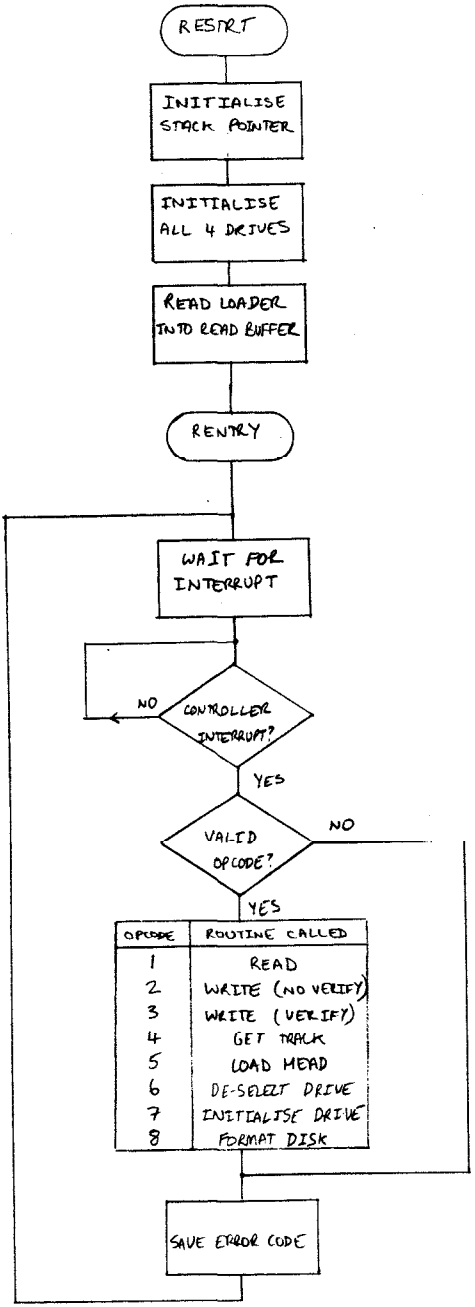


Figure 3.1 Flowchart for RESTRT and RENTRY

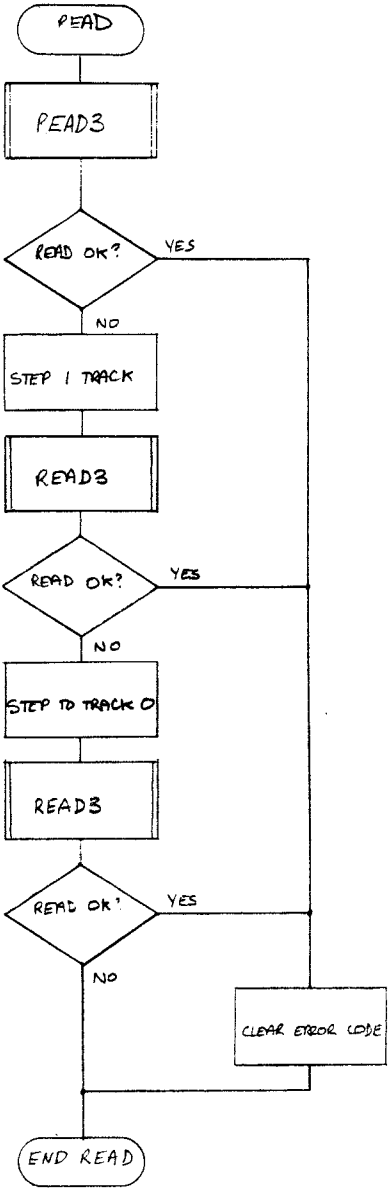


Figure 3.2 Flowchart for READ

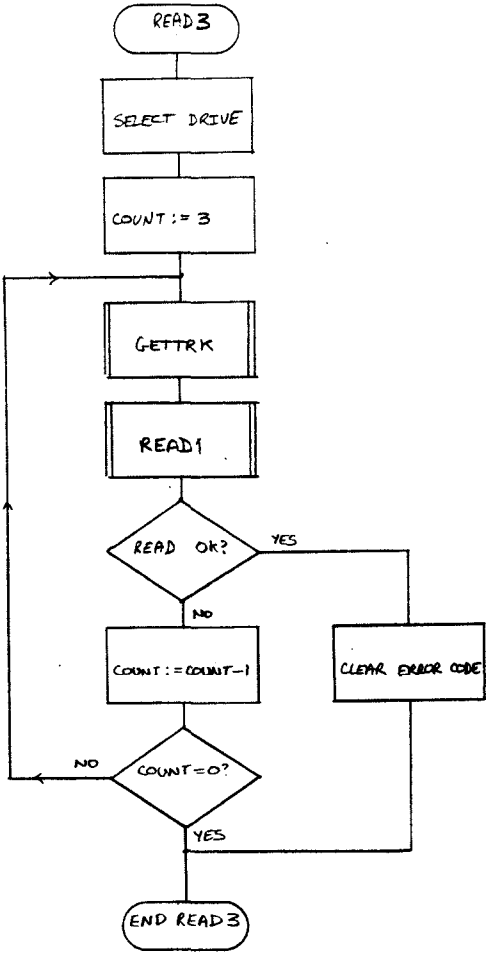


Figure 3.3 Flowchart for READ3

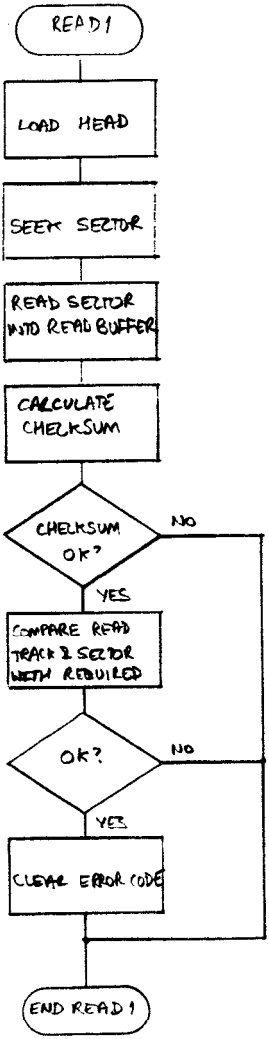


Figure 3.4 Flowchart for READ1

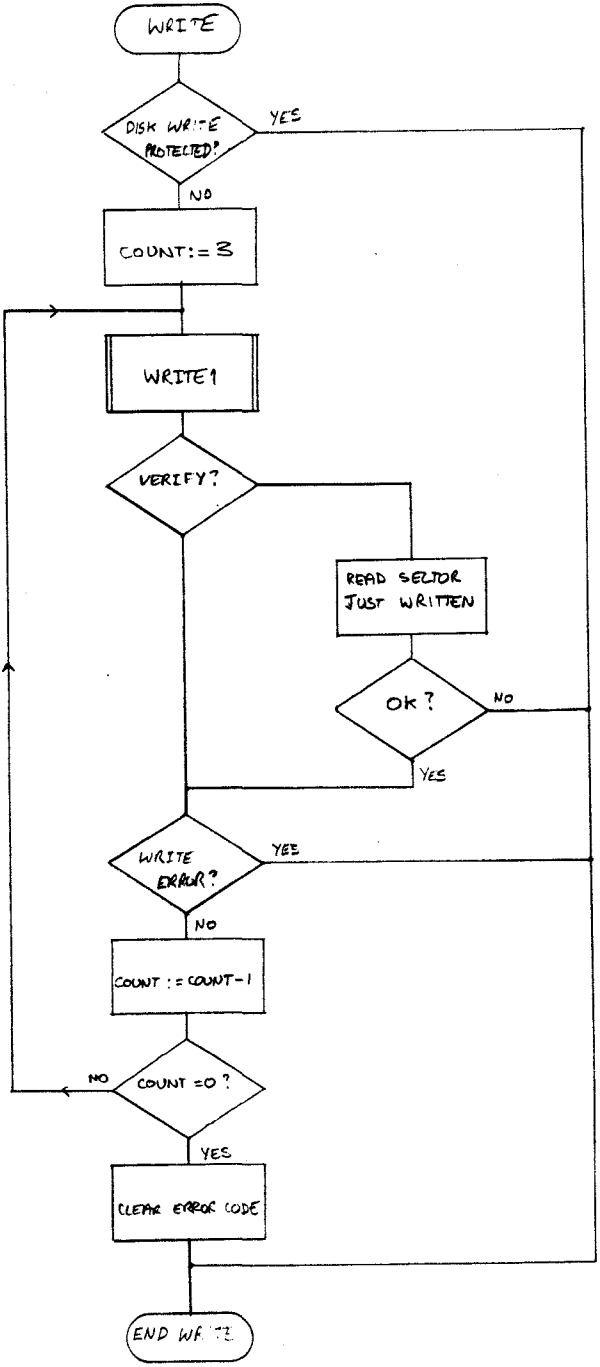


Figure 3.5 Flowchart for WRITE

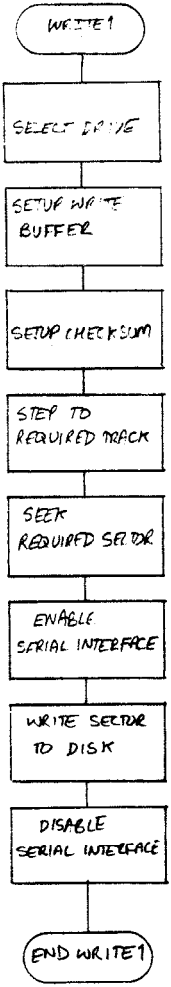


Figure 3.6 Flowchart for WRITE1

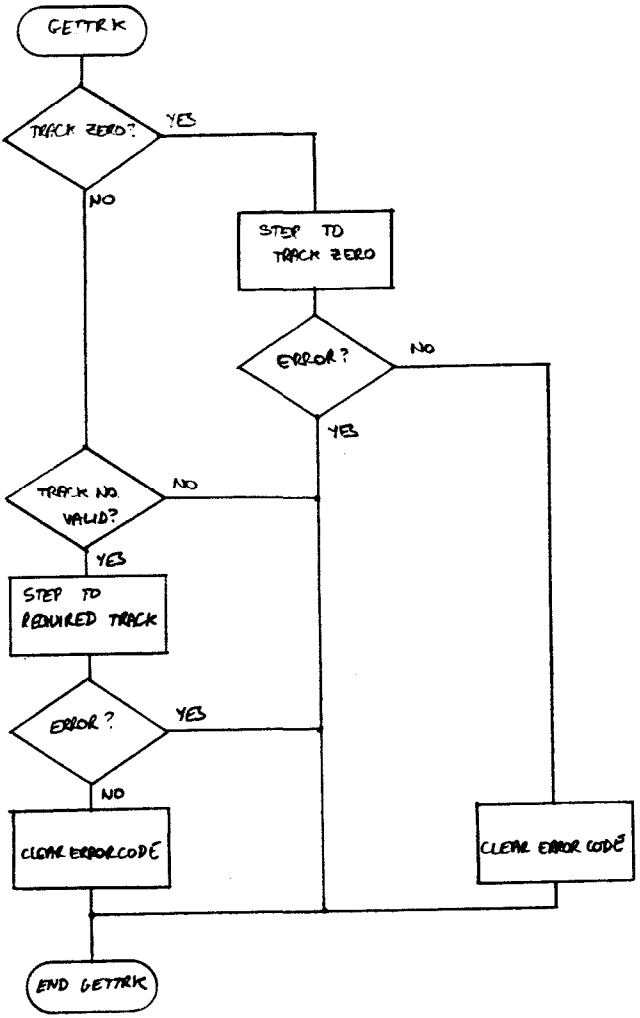


Figure 3.7 Flowchart for GETTRK

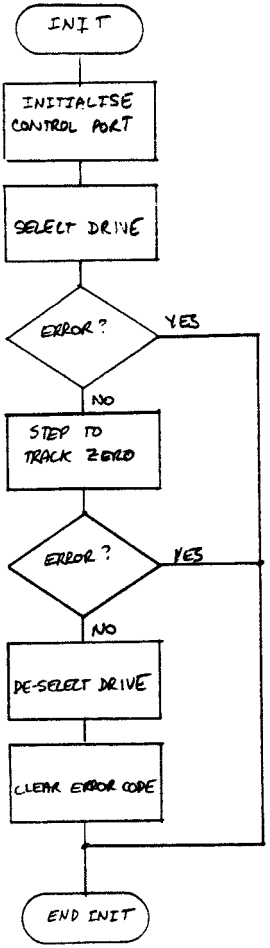


Figure 3.8 Flowchart for INIT

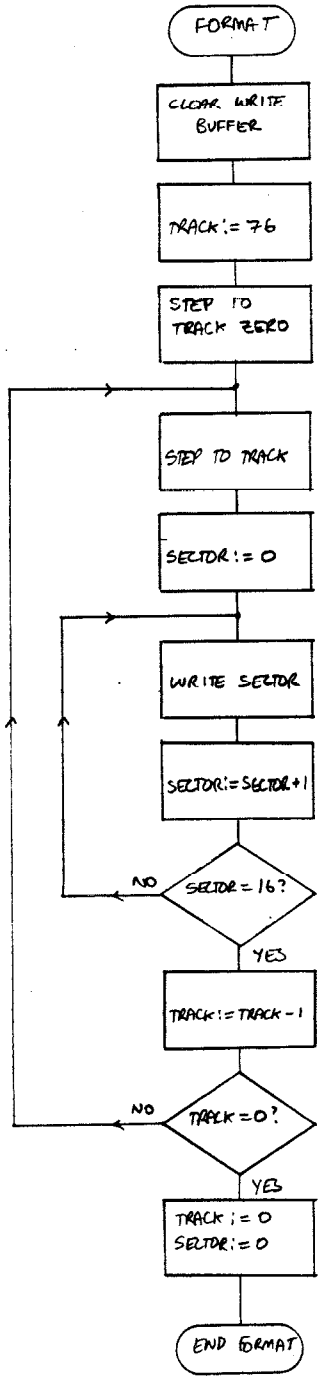


Figure 3.9 Flowchart for FORMAT

4/ Construction Information

The 77-68 disk controller board is a complex piece of equipment, and construction should not be undertaken by anyone without experience in soldering, since badly soldered joints or track shorts will certainly prevent correct operation of the board and may damage some of the more sensitive integrated circuits.

Constructional details are given step-by-step with testing details and testing programs. A multimeter is required for testing (minimum 10,000 ohms/volt and an oscilloscope could be useful if faults occur.

4.1./ Construction and part testing

(a) Insert all wire links. These should ideally be in thin insulated wire, although the long power supply runs may be in bare tinned copper wire. Once all the links have been inserted, GO BACK AND CHECK THEM ALL AGAIN! It is very easy to miss one, and it is far easier to find it now than when all of the components are inserted.

(b) Insert all sockets: These should be as follows:-

1 x 40 pin for IC21 (the 6800)
2 x 24 pin for IC22 (6852) and IC23 (2708)
2 x 18 pin for IC24 and IC25 (2114 or TMS40L45)

(c) Insert all passive components:-

Resistors R1 to R26
Capacitors C1 to C10
Diode D1
Transistors TR1

DO NOT INSERT THE 10MHz CRYSTAL YET

(d) Insert the following components:-

IC1	74LS04
IC4	74LS04
IC11	74LS27
IC12	74LS20
IC13	74LS30
IC14	74LS30
IC16	7437
IC17	74LS04

Insert the board into the 77-68 system, and enter the following program into the 77-68 memory at address 0100H:-

```
0100  B7 87FF
0103  7E 0100
```

Put the multimeter on the 0-10V range onto IC1 pin 10. The reading should be less than 0.4 volts. Execute the above program from address 0100. The voltmeter reading should rise about 0.8. volts. Operate the 77-68 RESET switch.

Change byte 0100H to B6. Put the multimeter on IC17 pin 10, and repeat the above test.

This has now checked the address decoding logic.

(e) Insert the following components:-

IC5	81LS97	
IC6	81LS97	
IC9	74LS157	
IC15	74LS00	
IC24	2114 (or TMS40L45)	into sockets
IC25	2114 (or TMS40L45)	

TAKE CARE WITH IC24 AND IC25 - They are easily damaged by careless handling.

Insert the board into the 77-68 system. Use the system monitor memory change command to change memory at address 8400H. If this works correctly, it shows that the RAM on the controller board can be accessed by the 77-68.

Now use the memory change command on address 87FFH. Try to change it to 5AH. This should fail, since this address of the RAM cannot be accessed as it is the address of the controller board status latch.

(f) Insert the following components:-

IC2	81LS97
IC10	74LS08
IC28	74LS74

Operate the 77-68 RESET switch. Use the monitor memory change command on address 87FFH. The current data should be FFH. Connect IC17 pin 9 to 0 volts, and release it. Re-read address 87FFH. It should now be 7FH. This has now checked all of the 77-68 interface logic.

(g) Insert the crystal. Insert the board into the 77-68 system and check the voltage on IC4 pin 12. It should be about 1 volt. This checks that the 10 MHz oscillator is working.

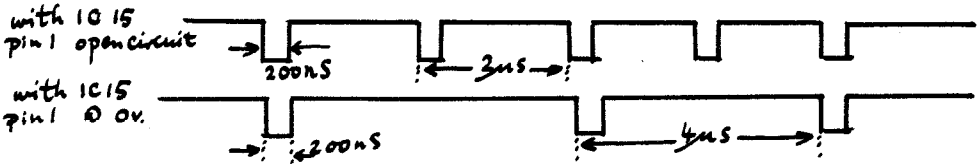
(h) Insert the following components:-

IC7	74LS290
IC8	74LS74
IC18	74LS160
IC26	74LS139
IC33	7402

Insert the board into the 77-68 system. Check the voltages on IC8 pin 9, IC33 pin 10 and IC33 pin 13. All should be about 1 volt. These check that the clock generator circuits are working.

Attach the voltmeter onto IC16 pin 8. The voltage should be greater than 2 volts, although the exact value doesn't matter. If IC15 pin 1 is shorted to 0V, the voltage on IC16 bin 8 should drop slightly. This checks the data out to the disk drive is switching correctly.

On an oscilloscope, the voltage on IC16 pin 8 will be as follows:-



(j) Now insert the remainder of components, leaving IC21, IC22, IC23 until last. CARE MUST BE TAKEN WITH THESE ICs AS THEY ARE STATIC-SENSITIVE. The components to be inserted are:-

IC3	81LS97
IC19	81LS97
IC20	74LS161
IC21	MC6800
IC22	MC6852
IC23	2708 with BEAR controller program
IC27	74LS175
IC29	74LS04
IC30	74LS138

IC31	74LS08
IC32	74LS32
IC34	74LS74
IC35	74LS123
IC36	7417
IC37	7417
IC38	74LS122
IC39	74LS175

The controller board should now be fully equipped. Before further testing however, the disk drive must be modified slightly by use of the user-selectable links on the board on the disk drive. This is to allow the read/write head to be loaded and unloaded indepently of the drive select signal.

On the DRI 7200 drive, this is done by removing the wire between pins 14 and 16 of IC header PP2, and connecting a wire between pins 14 and 3 of PP2.

On the DRI 7100 drive, this is done by removing the wire between HL and SI next to the edge connector and connecting a wire between HL, I/O Pin 18, and RHL.

Now insert the controller board into the 77-68 system, connect disk drive zero to both the controller board and to its power supplies, and insert a disk into the drive.

Operate the 77-68 RESET switch. The disk drive read/write head should now move to track zero, the head should load onto the disk, and then unload.

If this does not happen, then:-

(1) check all the power supplies to the disk drive are present, and correct (+5v, +24v, 240v AC)

(2) check all the pwer supplies to the controller board are correct (+5v, +12v, -12v)

(3) check all of the soldering on the board for short circuits between tracks, or unsoldered or badly soldered joints

(4) check all the wire links have been inserted

(5) check the disk was inserted correctly, and that the door was fully shut.

5/ Interfacing to the controller

To the 77-68 system the controller looks like a 1K byte block of memory between addresses 8400H and 87FFH. The parts of this 1K block which are required by a user program are as follows:-

842AH to 8529H - 256 byte write buffer
8559H to 8658H - 256 byte read buffer

8660H - Opcode for required operation (1-8)
8661H - Drive number to access (0-3)
8662H - Track number to access (0-76)
8663H - Sector number to access (0-15)
8664H - Returned error code (0-13)

These are only locations which the user need ever access. However, for debugging or ex-perimental purposes the other memory locations used are given in the assembly code and listing in Appendix C.

5.1 Opcodes

The following is a list of the operations which may be performed by the controller:-

Opcode = 1 - Read a sector into the read buffer
2 - Write a sector from the write buffer
3 - Write a sector and verify it
4 - Seek a track
5 - Load the read/write head onto the disk
6 - De-select the selected drive
7 - Initialise a drive
8 - Format a new disk

5.2. Parameters

The drive number is a value between 0 and 3 corresponding to drive 0 to 3. The drive number is required for all operations except for de-selecting the selected drive (opcode = 6)

The track number is a value between 0 and 76 corresponding to the 77 tracks on the disk. The track number is requred for reading and or writing a sector, and seeking a track (opcodes 1,2,3, and 4)

The sector number is a value between 0 and 15 corresponding to the 16 sectors on each track. The sector number is required for reading or writing a sector (opcodes 1,2,3)

If any values for the parameters other than those mentioned above are setup, an error will occur (if the parameter was required for the operation performed)

5.3. Controller Status

At address 87FFH is a command/status latch which is used for starting and checking the operation of the of the disk controller. Once the required buffers and parmeters have been set up, a write to address 87FFH will start the disk controller operation . Reading from address 87FFH will inform the user of the state of the controller. If bit 7 is zero, the controller is free. The controller RAM may only be accessed when bit 7 is zero.

5.4. Notes for users

The following additional notes are to help users of the 77-68 disk controller:-

(1) Never operate the 77-68 RESET switch when the disk read/write head is loaded onto the disk, unless it has been there for at least 10 seconds without moving.

(2) Always format a new disk before attempting to perform any read or write operations on it

(3) Whenever the RESET switch is operated, the controller will load track 0 sector 8 into the read buffer. This is intended for bringing a system loader off a disk to be executed, and bring the full system off the disk. This loader must be moved into user memory first, however.

Appendix A Error Numbers

After any operation of the disk controller has been completed, the error status at address 8664H should be checked. This will be zero if no error occurred in the last transfer, or will contain a value which gives the error which occurred. These errors are given below:-

00 No error detected
01 Track zero seek error
02 Attempt to select drive number > 3
03 Drive not ready or non-existent
04 Attempt to seek to track > 76
05 Attempt to write to write protected disk
06 Write error (write with verify only)
07 Attempt to seek to sector > 15
08 Checksum error on read
09 Track just read not same as requested track
10 Sector just read not same as requested sector
11 Invalid command
12 Disk transfer error
13 Track seek error

Processing on the disk controller will terminate as soon as an error is detected, and only the first error detected will be reported, so if more than one error exits a second error may occur when the first one is cured.

Appendix B Example Interface Routines

This appendix contains examples of software drivers for the disk controller board.

1/ Recommended Driver

This is the driver routine used in BEARDOS, the Disk operating system written to run with the 77-68 disk Controller board. It makes full use of the facilities of the controller board, and also has a built-in 3 second timeout in case the controller board fails to respond

```
;
; DISK CONTROLLER ADDRESSES

; OPCODE      EQU     8660H       ;REQUIRED OPERATION
; DRVWNT      EQU     8661H       ;REQUIRED DRIVE
; TRKWNT      EQU     8662H       ;REQUIRED TRACK-SECTOR
; ERCODE      EQU     8664H       ;RETURNED ERROR CODE
; DSTAT       EQU     87FFH       ;CONTROLLER STATUS LATCH
;
; ENTRY  = POINT
; ACCA   = DRIVE NUMBER
; ACCB   = CONTROLLER OP-CODE
; IX     = TRACK-SECTOR NUMBER
;
 DSKRUN: BSR     DSKEND    ;WAIT UNTIL CONTROLLER IS FREE
         STX     TRKWNT    ;SAVE TRACK-SECTOR
         STAB    OPCODE    ;SAVE CONTROLLER OP-CODE
         STAA    DRVWNT    ;SAVE DRIVE NUMBER
         STAA    DSTAT     ;START CONTROLLER OPERATION
         RTS
;
; GET DISK CONTROLLER FREE AND CHECK FOR ERRORS
;
 DSKEND: PSH   B           ; SAVE ACCB
         STX   SAVEIX      ; AND IX
         LDX   I,0
 TIMOUT: TST   DSTAT       ; GET CONTROLLER STATUS
         BPL   DISKND      ; IF BIT 7 = 0, CONTROLLER IS FREE
         BSR   RETN        ; ELSE DELAY
         BSR   RETN
         DEX               ; DECREMENT TIMEOUT COUNT
         BNE   TIMOUT
         LDAA  I, TIMERR   ; IF TIMEOUT COUNT REACHESZERO,
         BRA   DERROR      ; GO AND REPORT ERROR

;DISKND : LDAB  ERCODE     ; IF CONTROLLER FREE, GET ERROR CODE
         CLR   ERCODE      ; CLEAR ERROR LOCATION ON CONTROLLER
         TSTB              ; TEST ERROR CODE
         BNE   DISKER      ; IF NON ZERO, GO AND REPORT ERROR
```

```
            LDX     SAVEIX  ;ELSE RESTORE IX
            PULB            ;AND ACCB
RETN        RTS
;
DISKER      LDAA    I,DSKERR;ERROR - DISK CONTROLLER ERROR
;
DERROR      INS             REMOVE ACCB FROM STACK
            :
            Disk error reporting routine
            (Error reporting routine number in ACCA, Controller
            error in code ACCB)
```

This recommended driver is 55 bytes long excluding the error reporting routine but it does ensure that the software in the 77-68 can never get totally locked up, and it also has comprehensive reporting of errors. In addition, once an operation has started, the 77-68 software continues processing until it requires to access the controller board again.

2/ SIMPLE DRIVER

The simpler Driver is only 32 bytes long but does not have such comprehensive fault testing as the recommended driver.

```
;
;ENTRY POINT
;
DSKRUN: BSR     DSKEND      ;WAIT UNTIL CONTROLLER IS FREE
        STX     TRKWNT      ;SAVE TRACK-SECTOR
        STAB    OPCODE      ;SAVE CONTROLLER OP-CODE
        STAA    DRVWNT      ;SAVE DRIVE NUMBER
        STAA    DSTAT       ;START CONTROLLER OPERATION
        RTS                 ;
DSKEND: PSHB                ;SAE ACCB
DLOOP : TST     DSTAT       ;TEST CONTROLLER STATUS
        BMI     DLOOP       ;LOOP UNTIL BIT 7 = 0
        LDAB    ERCODE      ;WHEN CONTROLLER FREE, GET ERROR CODE
        BNE     DERROR      ;IF NON-ZERO, HAVE ERROR
        PULB                ;ELSE RESTORE ACCB
        RTS
DERROR: INS                 ;IF ERROR
        CLR ERCODE
        :
        REPORT ERROR        (Controller error number in ACCB)
```

In the case of either of the above driver routines if it is required to access the read or write buffers on the controller board, the sub-routine DSKEND should be called first to ensure that the controller is free and that it can be accessed by the 77-68

```
ASM80 :F1:FLOPPY.CTL NOSYMBOLS

ISIS-II 8080/8085 MACRO ASSEMBLER, V2.0        MODULE   PAGE    1

  LOC  OBJ                SOURCE STATEMENT

                        $MACROFILE
                      = $INCLUDE(:F1:M6800.DEF)
                      = $NOPAGING NOCOND SAVE LIST
                      = ;
                      = ;  CROSS-ASSEMBLER FOR MOTOROLA 6800, V1.0
                      = ;  ======================================
                      = ;
                      = ;****  DO NOT USE COND/NOCOND  ****
                      = ;****  DO NOT USE  GEN/NOGEN   ****
                      = ;****  DO NOT USE RELOCATION   ****
                      = ;****  DO NOT USE MACROS       ****
                      = ;****  DEFINE DATA AREAS FIRST ****
                      = ;
                      = $NOLIST
                      = $RESTORE
                        ;
                        ;
                        ;
                        ;FLOPPY DISK DRIVER ROUTINES V0.0
                        ;CONTROLLER CARD
                        ;WRITTEN BY DAVE HOWLAND
                        ;COPYRIGHT 1978
                        ;
                        ;
                        ;
                        ;
  0000                          ORG     0
                        ;
  0140                  WRTBUF: RMB     140H            ;WRITE BUFFER
  0120                  RDBUFF: RMB     120H            ;READ BUFFER
  0001                  OPCODE: RMB     1               ;INFORMATION INPUT BY HOST PROCESSOR-
  0001                  DRVNMT: RMB     1               ;OP-CODE, DRIVE, TRACK, SECTOR
  0001                  TRKNMT: RMB     1
  0001                  SCTNMT: RMB     1
  0001                  ERCODE: RMB     1               ;ERROR CODE RETURNED TO HOST PROCESSOR
  0001                  CONTRL: RMB     1               ;IMAGE OF CONTROL PORT
  0001                  LOOPCT: RMB     1
  0004                  COUNTO: RMB     4               ;STORAGE FOR CURRENT TRACK FOR 4 DRIVES
  0002                  BUFLEN: RMB     2
  0001                  RDCNT:  RMB     1
  0001                  WLOOP:  RMB     1
  0001                  BYTCNT: RMB     1
  0001                  CHKSTS: RMB     1               ;VERIFY WRITE STATUS
                        ;
  03C0                          ORG     3C0H
```

```
                        *
0040                    MSTACK: RMB     40H             *CONTROL PROCESSOR STACK
                        *
                        *
                        *STACK POINTER
                        *
03F7                    STKPTR  EQU     3F7H            *INITIALISE STACK POINTER
                        *
                        *
                        *WRITE BUFFER
                        *
0010                    SYNBYT  EQU     WRTBUF+10H      *DATA SYNC BYTE
0011                    WBUFST  EQU     WRTBUF+11H      *START OF BUFFER PROPER
001C                    TRKNUM  EQU     WRTBUF+1CH      *TRACK NUMBER OF CURRENT BLOCK
001D                    SCTNUM  EQU     WRTBUF+1DH      *SECTOR NUMBER OF CURRENT BLOCK
002A                    WRDATA  EQU     WRTBUF+2AH      *START OF WRITE DATA
012A                    WCKSUM  EQU     WRTBUF+12AH     *CHECKSUM
012B                    WBUFND  EQU     WRTBUF+12BH     *END OF WRITE BUFFER
                        *
                        *
                        *READ BUFFER
                        *
014B                    TRAKIN  EQU     RDBUFF+0BH      *TRACK NUMBER OF CURRENT BLOCK
014C                    SECTIN  EQU     RDBUFF+0CH      *SECTOR NUMBER OF CURRENT BLOCK
0159                    RDATA   EQU     RDBUFF+19H      *START OF READ DATA
0259                    RCKSUM  EQU     RDBUFF+119H     *CHECKSUM
025A                    RBUFND  EQU     RDBUFF+11AH     *END OF READ BUFFER
                        *
                        *
                        *I/O PORT ADDRESSES
                        *
4010                    SSDAC1  EQU     4010H           *SERIAL INTERFACE PORT
4010                    SSDAST  EQU     4010H
4011                    SSDAOP  EQU     4011H
4011                    SSDAIP  EQU     4011H
4011                    SSDACR  EQU     4011H
8000                    CTLOUT  EQU     8000H           *CONTROL PORT (OUTPUT)
8000                    STATUS  EQU     8000H           *STATUS PORT (INPUT)
                        *
                        *
                        *SSDA DATA
                        *
000B                    SELC2   EQU     00001011B       *SERIAL INTERFACE PORT INITIALISATION
004B                    SELC3   EQU     01001011B
008B                    SELSYN  EQU     10001011B
00CB                    RESET   EQU     11001011B       *SERIAL INTERFACE PORT MASTER RESET
001C                    SETC2   EQU     00011100B
000E                    SETC3   EQU     00001110B
00B1                    SYNC    EQU     10000001B       *DATA SYNC WORD
00C2                    SYNSCH  EQU     11000010B       *DATA SYNC SEARCH CONTROL WORD
00C9                    TRANEN  EQU     11001001B       *ENABLE TRANSMIT SECTION
0001                    RDA     EQU     00000001B       *RECEIVE DATA AVAILABLE MASK
0002                    TDRA    EQU     00000010B       *TRANSMIT DATA REGISTER AVAILABLE MASK
                        *
                        *
                        *ERROR NUMBERS
```

```
                        ;
0001                    TKZERR  EQU     01H             ;TRACK ZERO SEEK ERROR
0002                    INVALD  EQU     02H             ;ATTEMPT TO SELECT DRIVE > 3
0003                    DUNRDY  EQU     03H             ;DRIVE NOT READY OR NON-EXISTANT
0004                    TRKERR  EQU     04H             ;ATTEMPT TO SEEK TRACK > 76
0005                    WPRTCT  EQU     05H             ;ATTEMPT TO WWRITE TO WRITE-PROTECTED DISK
0006                    WRTERR  EQU     06H             ;FAIL 3 TIMES TO WRITE CORRECTLY TO DISK
0007                    SCTERR  EQU     07H             ;ATTEMPT TO SEEK SECTOR > 15
0008                    CKSMER  EQU     08H             ;CHECKSUM ERROR ON READ
0009                    TRKER   EQU     09H             ;TRACK JUST READ NOT SAME AS REQUESTED TRACK
0010                    SECER   EQU     10H             ;SECTOR JUST READ NOT SAME AS REQUESTED SECTOR
0011                    NOCMND  EQU     11H             ;COMMAND INVALID
0012                    DSKIO   EQU     12H             ;DISK I/O ERROR
0013                    TKSKER  EQU     13H             ;TRACK SEEK ERROR
                        ;
                        ;
                        ;DELAY CONSTANTS
                        ;
02EE                    DEL6    EQU     750             ;CONSTANT FOR 6MSEC DELAY
06D6                    DEL14   EQU     1750            ;CONSTANT FOR 14MSEC DELAY
0EA6                    DEL30   EQU     3750            ;CONSTANT FOR 30MSEC DELAY
                        ;
                        ;
                        ;CONTROL FUNCTION MASKS
                        ;
0080                    UNLOAD  EQU     10000000B       ;UNLOAD HEAD
007F                    NOTULD  EQU     01111111B
0040                    HEDLOD  EQU     01000000B       ;LOAD HEAD
00BF                    NOTLOD  EQU     10111111B
0020                    WGATON  EQU     00100000B       ;WRITE GATE ON
00DF                    WGTOFF  EQU     11011111B       ;WRITE GATE OFF
0010                    DIN     EQU     00010000B       ;SET HEAD STEP DIRECTION TO IN
00EF                    DOUT    EQU     11101111B       ;SET HEAD STEP DIRECTION TO OUT
0008                    STEPHI  EQU     00001000B       ;SET STEP PULSE HIGH
00F7                    STEPLO  EQU     11110111B       ;SET STEP PULSE LOW
0004                    DRIVEN  EQU     00000100B       ;ENABLE SELECTED DRIVE
00FB                    DSELCT  EQU     11111011B       ;DISABLE DRIVE
0003                    DSKNUM  EQU     00000011B       ;DRIVE NUMBER
00FC                    SELDVO  EQU     11111100B       ;SELECT DRIVE ZERO
                        ;
                        ;
                        ;STATUS WORD MASKS
                        ;
0080                    HDSTAT  EQU     10000000B       ;HEAD LOAD STATUS
0040                    TRZERO  EQU     01000000B       ;TRACK ZERO
0020                    READY   EQU     00100000B       ;DRIVE READY
0010                    WRTPRT  EQU     00010000B       ;WRITE PROTECT
000F                    SECTOR  EQU     00001111B       ;SECTOR NUMBER
                        ;
                        ;LOADER TRACK-SECTOR
                        ;
0008                    LOADTS  EQU     0008H
                        ;
                        ;
                        ;
                        ;SET BASE ADDRESS OF PROGRAM
```

```
                        *
FC00                    ORG     0FC00H
                        *
                        *
                        *INITIALISE THE SELECTED DRIVE
                        *
FC00 8D04               INIT:   BSR     INIT1           *ENTRY POINT FOR EXTERNAL INITIALISE CALL
FC02 2501               BCS     ERROR1
FC04 5F                 CLRB
FC05 39                 ERROR1: RTS
FC06 5F                 INIT1:  CLRB                    *INTERNAL ENTRY POINT
FC07 8D41               BSR     OUTCTL          *CLEAR CONTROL PORT AND CONTROL BYTE
FC09 BDFCE0             JSR     HDUNLD          *CLEAR HEAD LOAD MONOSTABLE
FC0C 8D4A               BSR     DSKSEL          *SELECT REQUIRED DRIVE
FC0E 2507               BCS     INITEX          *BRANCH IF ERROR
FC10 8D06               BSR     TRKZER          *SEEK TO TRACK ZERO
FC12 2503               BCS     INITEX          *BRANCH IF ERROR
FC14 BDFCEE             JSR     DISSEL          *DESELECT DRIVE
FC17 39                 INITEX: RTS
                        *
                        *
                        *FIND TRACK ZERO, AND RESET THE TRACK COUNTER
                        *
FC18 B68000             TRKZER: LDAA    STATUS          *LOAD STATUS BYTE
FC1B 8540               BITA    I,TRZERO        *TEST TRACK ZERO BIT
FC1D 261F               BNE     ZREADY          *IF ALREADY AT TRACK ZERO, EXIT
FC1F 8664               LDAA    I,100
FC21 B70266             STAA    LOOPCT          *MOVE VALUE 100 INTO LOOP COUNT
FC24 8D2B               BSR     DIROUT          *SET STEP DIRECTION TO OUT
FC26 B68000             TZLOOP: LDAA    STATUS          *LOAD STATUS BYTE
FC29 8540               BITA    I,TRZERO        *TEST TRACK ZERO BIT
FC2B 260B               BNE     ZFOUND          *EXIT IF AT TRACK ZERO
FC2D 8D59               BSR     STEP            *IF NOT, STEP OUT ONE TRACK
FC2F 7A0266             DEC     LOOPCT          *DECREMENT LOOP COUNT
FC32 26F2               BNE     TZLOOP          *IF LOOP COUNT REACHES ZERO, HAVE ERROR,
FC34 C601               LDAB    I,TKZERR        *AS MAXIMUM NUMBER OF STEPS SHOULD BE 77
FC36 0D                 SEC                     *SET CARRY BIT (TO INDICATE ERROR)
FC37 39                 RTS
FC38 CE06D6             ZFOUND: LDX     I,DEL14         *DELAY 14MSEC TO ALLOW HEAD TO SETTLE
FC3B 09                 ZDELAY: DEX
FC3C 26FD               BNE     ZDELAY
FC3E BDFCF8             ZREADY: JSR     GETCNT          *GET POINTER TO CURRENT TRACK NUMBER
FC41 6F00               CLR     0,X             *AND SET TRACK NUMBER TO ZERO
FC43 0C                 CLC                     *CLEAR CARRY (NO ERROR)
FC44 39                 RTS
                        *
                        *
                        *SET DIRECTION BIT TO 'IN'
                        *
FC45 F60265             DIRIN:  LDAB    CONTRL          *LOAD CONTROL BYTE
FC48 CA10               ORAB    I,DIN           *SET DIRECTION BIT HIGH
FC4A F70265             OUTCTL: STAB    CONTRL          *STORE MODIFIED CONTROL BYTE
FC4D F78000             STAB    CTLOUT          *OUTPUT TO CONTROL PORT
FC50 39                 RTS
                        *
                        *
```

```
                        ;SET DIRECTION BIT TO 'OUT'
                        ;
FC51 F60265     DIROUT: LDAB    CONTRL          ;LOAD CONTROL BYTE
FC54 C4EF               ANDB    I,DOUT          ;SET DIRECTION BIT LOW
FC56 20F2               BRA     OUTCTL          ;STORE AND OUTPUT MODIFIED CONTROL BYTE
                        ;
                        ;
                        ;SELECT REQUIRED DISC DRIVE
                        ;
FC58 F60261     DSKSEL: LDAB    DRVWNT          ;LOAD REQUIRED DRIVE NUMBER
FC5B C103               CMPB    I,3
FC5D 2221               BHI     NOTDRV          ;IF DRIVE NUMBER > 3, HAVE ERROR
FC5F B60265             LDAA    CONTRL          ;LOAD CONTROL BYTE
FC62 8403               ANDA    I,DSKNUM        ;MASK OFF 6 MSB TO GET CURRENT DRIVE
FC64 11                 CBA
FC65 2708               BEQ     DRVSAM          ;IF CURRENT = REQUIRED, BRANCH TO ENABLE ROUTINE
FC67 8D77               BSR     HDUNLD          ;IF NOT, UNLOAD HEAD
FC69 B60265             LDAA    CONTRL          ;LOAD CONTROL BYTE
FC6C 84FC               ANDA    I,SELDVO        ;CLEAR DRIVE NUMBER BITS
FC6E 1B                 ABA                     ;SETUP REQUIRED DRIVE NUMBER
FC6F 8A04       DRVSAM: ORAA    I,DRIVEN        ;SET DRIVE ENABLE BIT HIGH
FC71 B78000             STAA    CTLOUT          ;OUTPUT TO CONTROL PORT
FC74 B70265             STAA    CONTRL          ;AND STORE
FC77 B68000             LDAA    STATUS          ;LOAD STATUS BYTE
FC7A 8520               BITA    I,READY         ;TEST DRIVE READY BIT
FC7C 2706               BEQ     NOTRDY          ;IF NOT READY, HAVE ERROR
FC7E 0C         ENDSEL: CLC
FC7F 39                 RTS
FC80 C602       NOTDRV: LDAB    I,INVALD        ;ERROR - DRIVE NUMBER INVALID
FC82 0D                 SEC
FC83 39                 RTS
FC84 C603       NOTRDY: LDAB    I,DUNRDY        ;ERROR - DRIVE NOT READY OR NON-EXISTANT
FC86 0D                 SEC
FC87 39                 RTS
                        ;
                        ;
                        ;STEP ONE TRACK IN SELECTED DIRECTION
                        ;
FC88 37         STEP:   PSHB
FC89 F60265             LDAB    CONTRL          ;LOAD CONTROL BYTE
FC8C CA08               ORAB    I,STEPHI        ;SET STEP BIT HIGH
FC8E F78000             STAB    CTLOUT          ;OUTPUT TO CONTROL PORT
FC91 C4F7               ANDB    I,STEPLO        ;SET STEP BIT LOW
FC93 F78000             STAB    CTLOUT          ;OUTPUT TO CONTROL PORT
FC96 8D60               BSR     GETCNT          ;GET POINTER TO CURRENT TRACK NUMBER
FC98 C510               BITB    I,DIN           ;TEST DIRECTION BIT
FC9A 2604               BNE     STEPIN
FC9C 6A00               DEC     0,X             ;IF LOW, DECREMENT CURRENT TRACK NUMBER
FC9E 2002               BRA     STEPEX
FCA0 6C00       STEPIN: INC     0,X             ;IF HIGH, INCREMENT CURRENT TRACK NUMBER
FCA2 CE02EE     STEPEX: LDX     I,DEL6          ;DELAY 6MSEC (TRACK-TO-TRACK DELAY)
FCA5 09         SDELAY: DEX
FCA6 26FD               BNE     SDELAY
FCA8 33                 PULB
FCA9 39                 RTS
                        ;
```

```
                        *
                        *GET THE REQUIRED TRACK
                        *
FCAA B60262     GETTRK: LDAA    TRKWNT          *LOAD REQUIRED TRACK NUMBER
FCAD 2603               BNE     NOTZER
FCAF 7EFC18             JUMP    TRKZER          *IF ZERO, GO TO TRACK ZERO ROUTINE
FCB2 814C       NOTZER: CMPA    I,76
FCB4 2222               BHI     NOTRAK          *IF REQUIRED TRACK NUMBER > 76, HAVE ERROR
FCB6 8D40               BSR     GETCNT          *IF NOT, GET POINTER TO CURRENT TRACK NUMBER
FCB8 A000               SUBA    0,X             *SUBTRACT CURRENT FROM REQUIRED TRACK NUMBER
FCBA 2719               BEQ     TRKSAM          *IF ZERO, CURRENT = REQUIRED TRACK - EXIT ROUTINE
FCBC 2B04               BMI     STPOUT
FCBE 8D85               BSR     DIRIN           *IF POSITIVE, SET DIRECTION BIT TO IN
FCC0 2003               BRA     STEP1
FCC2 8D8D       STPOUT: BSR     DIROUT          *IF NEGATIVE, SET DIRECTION BIT TO IN
FCC4 40                 NEGA                    *AND 2'S COMPLEMENT ACCA
FCC5 C64D       STEP1:  LDAB    I,77            *SETUP TRACK SEEK SAFETY COUNT
FCC7 8D8F       STPLOP: BSR     STEP            *STEP ONCE IN REQUIRED DIRECTION
FCC9 5A                 DECB                    *DECREMENT SAFETY COUNT
FCCA 2710               BEQ     GETKER          *IF ZERO, HAVE TRACK SEEK ERROR
FCCC 4A                 DECA                    *DECREMENT ACCA (STEP COUNTER)
FCCD 26F8               BNE     STPLOP          *IF ACCA <> 0, STEP AGAIN
FCCF CE06D6             LDX     I,DEL14         *DELAY 14MSEC
FCD2 09         GDELAY: DEX
FCD3 26FD               BNE     GDELAY
FCD5 5F         TRKSAM: CLRB                    *CLEAR ERROR STATUS
FCD6 0C                 CLC
FCD7 39                 RTS
FCD8 C604       NOTRAK: LDAB    I,TRKERR        *ERROR - TRACK NUMBER INVALID
FCDA 0D                 SEC
FCDB 39                 RTS
FCDC C613       GETKER: LDAB    I,TKSKER        *ERROR - TRACK SEEK ERROR
FCDE 0D                 SEC
FCDF 39                 RTS
                        *
                        *
                        *UNLOAD THE HEAD
                        *
FCE0 B60265     HDUNLD: LDAA    CONTRL          *LOAD CONTROL BYTE
FCE3 8A80               ORAA    I,UNLOAD        *SET UNLOAD BIT HIGH
FCE5 B78000             STAA    CTLOUT          *OUTPUT TO CONTROL PORT
FCE8 847F               ANDA    I,NOTULD        *SET UNLOAD BIT LOW
FCEA B78000             STAA    CTLOUT          *OUTPUT TO CONTROL PORT
FCED 39                 RTS
                        *
                        *
                        *DE-SELECT THE DISC DRIVE
                        *
FCEE F60265     DISSEL: LDAB    CONTRL          *LOAD CONTROL BYTE
FCF1 C4FB               ANDB    I,DSELCT        *SET DRIVE ENABLE BIT LOW
FCF3 BDFC4A             JSR     OUTCTL          *OUTPUT TO CONTROL PORT, AND STORE IN MEMORY
FCF6 5F                 CLRB
FCF7 39                 RTS
                        *
                        *
                        *SET POINTER TO CURRENT TRACK OF SELECTED DRIVE
```

```
                        *
FCF8 37         GETCNT: PSHB                    *SAVE ACCB
FCF9 CE0267             LDX     I,COUNTO        *GET POINTER TO CURRENT TRACK NUMBER
FCFC F60261             LDAB    DRVWNT          *LOAD REQUIRED DRIVE NUMBER
FCFF 5D         CNTLOP: TSTB                    *TEST ACCB
FD00 2704               BEQ     GCOUNT          *IF ZERO, EXIT
FD02 5A                 DECB                    *IF NOT, DECREMENT ACCB
FD03 08                 INCX                    *AND INCREMENT POINTER
FD04 20F9               BRA     CNTLOP
FD06 33         GCOUNT: PULB
FD07 0C                 CLC
FD08 39                 RTS
                        *
                        *
                        *LOAD THE HEAD
                        *
FD09 F68000     HDLOAD: LDAB    STATUS          *LOAD STATUS BYTE
FD0C B60265             LDAA    CONTRL          *LOAD CONTROL BYTE
FD0F 8A40               ORAA    I,HEDLOD        *SET HEAD LOAD BIT HIGH
FD11 B78000             STAA    CTLOUT          *OUTPUT TO CONTROL PORT
FD14 84BF               ANDA    I,NOTLOD        *SET HEAD LOAD BIT LOW
FD16 B78000             STAA    CTLOUT          *OUTPUT TO CONTROL PORT
FD19 C580               BITB    I,HDSTAT        *TEST HEAD STATUS BEFORE LOADING
FD1B 2607               BNE     HDRET           *IF HEAD ALREADY LOADED, EXIT
FD1D CE0EA6             LDX     I,DEL30         *IF NOT, DELAY 30MSEC (HEAD LOAD TIME)
FD20 09         HDELAY: DEX
FD21 26FD               BNE     HDELAY
FD23 5F                 CLRB
FD24 39         HDRET:  RTS
                        *
                        *
                        *TRY 3 TIMES TO WRITE A
                        *256 BYTE BLOCK TO THE DISK
                        *
FD25 860F       VWRITE: LDAA    I,0FH
FD27 B70270             STAA    CHKSTS          *SETUP FOR WRITE VERIFY
FD2A 2003               BRA     WRITEX
                        *
                        *
                        *WRITE BLOCK TO DISK WITHOUT VERIFY
                        *
FD2C 7F0270     WRITE:  CLR     CHKSTS          *SETUP FOR NO WRITE VERIFY
FD2F B68000     WRITEX: LDAA    STATUS          *LOAD STATUS BYTE
FD32 8510               BITA    I,WRTPRT        *TEST WRITE PROTECT BIT
FD34 2703               BEQ     NOTPRT
FD36 C605               LDAB    I,WPRTCT        *ERROR - DISK WRITE PROTECTED
FD38 39                 RTS
FD39 8603       NOTPRT: LDAA    I,3
FD3B B7026E             STAA    WLOOP           *LOAD WLOOP WITH NUMBER OF WRITE ATTEMPTS
FD3E 8D11       WRITE2: BSR     WRITE1          *TRY ONE WRITE TO THE DISK
FD40 250E               BCS     WERROR
FD42 BDFE0C             JSR     CHKWRT          *CHECK THAT DATA WRITTEN CORRECTLY
FD45 2408               BCC     WRITOK          *IF OK, EXIT
FD47 7A026E             DEC     WLOOP           *IF NOT, DECREMENT WLOOP
FD4A 26F2               BNE     WRITE2          *IF WLOOP <> 0, TRY ANOTHER WRITE
FD4C C606               LDAB    I,WRTERR        *ERROR - DATA FAILED TO WRITE CORRECTLY TO DISK
```

```
FD4E 39              RTS
FD4F 5F        WRITOK: CLRB
FD50 39        WERROR: RTS
                       *
                       *
                       *TRY ONCE TO WRITE A
                       *256 BYTE BLOCK TO THE DISK
                       *
FD51 BDFC58    WRITE1: JSR     DSKSEL          *SELECT REQUIRED DRIVE
FD54 2558              BCS     WERR1
FD56 CE0000            LDX     I,WRTBUF        *GET POINTER TO WRITE BUFFER
FD59 8D67              BSR     CLKSYN          *SETUP LEADING CLOCK SYNC (16 BYTES OF ZEROS)
FD5B CE012C            LDX     I,WBUFND+1      *GET POINTER TO END OF WRITE BUFFER
FD5E 8D62              BSR     CLKSYN          *SETUP CLOCK OVER-RUN (16 BYTES OF ZEROS)
FD60 FE0262            LDX     TRKWNT
FD63 DF1C              STX     TRKNUM          *MOVE TRACK AND SECTOR NUMBER TO WRITE BUFFER
FD65 CE0011            LDX     I,WBUFST        *GET POINTER TO START OF BUFFER PROPER
FD68 8D61              BSR     CKSUM           *CALCULATE CHECKSUM
FD6A F7012A            STAB    WCKSUM          *SAVE AT END OF DATA BLOCK
FD6D B7012B            STAA    WCKSUM+1
FD70 8D12              BSR     WSETUP          *SETUP DISK DRIVE FOR WRITE TRANSFER
FD72 253A              BCS     WERR1
FD74 8D39              BSR     WRTLOP          *WRITE DATA ONTO DISK
                       *
                       *
                       *END WRITE TRANSFER
                       *
FD76 F60265    WCLOSE: LDAB    CONTRL          *LOAD CONTROL BYTE
FD79 C4DF              ANDB    I,WGTOFF        *SET WRITE GATE BIT LOW
FD7B BDFC4A            JSR     OUTCTL          *OUTPUT TO CONTROL PORT AND STORE IN MEMORY
FD7E 86CB              LDAA    I,RESET
FD80 B74010            STAA    SSDAC1          *RESET SERIAL INTERFACE
FD83 39                RTS
                       *
                       *
                       *SETUP FOR WRITE TRANSFER
                       *
FD84 8681      WSETUP: LDAA    I,SYNC
FD86 9710              STAA    SYNBYT          *MOVE DATA SYNC WORD INTO BUFFER
FD88 BDFCAA            JSR     GETTRK          *SEEK REQUIRED TRACK
FD8B 2521              BCS     WERR1
FD8D BDFD09            JSR     HDLOAD          *LOAD HEAD
FD90 B60265            LDAA    CONTRL          *LOAD CONTROL BYTE
FD93 8A20              ORAA    I,WGATON        *SET WRITE GATE BIT HIGH
FD95 B70265            STAA    CONTRL
FD98 5F                CLRB
FD99 F74011            STAB    SSDAOP          *MOVE 2 ZERO BYTES INTO SERIAL INTERFACE FIFO
FD9C F74011            STAB    SSDAOP
FD9F C6C9              LDAB    I,TRANEN        *LOAD ACCB WITH TRANSMIT ENABLE CODE
FDA1 CE0000            LDX     I,WRTBUF        *LOAD IX WITH START OF WRITE BUFFER
FDA4 8D41              BSR     GETSCT          *SEEK REQUIRED SECTOR
FDA6 2506              BCS     WERR1
FDA8 B78000            STAA    CTLOUT          *ENABLE WRITE GATE
FDAB F74010            STAB    SSDAC1          *ENABLE TRANSMIT
FDAE 39        WERR1:  RTS
                       *
```

```
                        *
                        *WRITE DATA TO DISK
                        *
FDAF A600       WRTLOP: LDAA    0,X             *LOAD BYTE FROM BUFFER
FDB1 08                 INCX                    *INCREMENT BUFFER POINTER
FDB2 C602       NREADY: LDAB    I,TDRA          *TEST TRANSMIT DATA REGISTER AVAILABLE
FDB4 F54010             BITB    SSDAST
FDB7 27F9               BEQ     NREADY
FDB9 B74011             STAA    SSDAOP          *IF AVAILABLE, OUTPUT BYTE TO SERIAL INTERFACE
FDBC 8C013B             CPX     I,WBUFND+16     *TEST FOR END OF BUFFER
FDBF 26EE               BNE     WRTLOP
FDC1 39                 RTS
                        *
                        *
                        *SETUP CLOCK SYNC
                        *
FDC2 8610       CLKSYN: LDAA    I,16
FDC4 6F00       CLOOP:  CLR     0,X             *CLEAR MEMORY LOCATION
FDC6 08                 INCX                    *INCREMENT POINTER
FDC7 4A                 DECA                    *DECREMENT COUNTER
FDC8 26FA               BNE     CLOOP
FDCA 39                 RTS
                        *
                        *
                        *CALCULATE A 16 BIT CHECKSUM
                        *
FDCB C602       CKSUM:  LDAB    I,2
FDCD F7026B             STAB    BUFLEN
FDD0 C618               LDAB    I,24
FDD2 F7026C             STAB    BUFLEN+1        *MOVE NUMBER OF BYTES TO BUFFER LENGTH COUNTER
FDD5 5F                 CLRB
FDD6 4F                 CLRA
FDD7 AB00       CKSUM1: ADDA    0,X             *CALCULATE 16 BIT CHECKSUM
FDD9 C900               ADCB    I,0             *ACCB = MSBYTE, ACCA = LSBYTE
FDDB 08                 INCX                    *INCREMENT POINTER
FDDC 7A026C             DEC     BUFLEN+1
FDDF 26F6               BNE     CKSUM1
FDE1 7A026B             DEC     BUFLEN          *DECREMENT COUNTER
FDE4 26F1               BNE     CKSUM1
FDE6 39                 RTS
                        *
                        *
                        *GET THE REQUIRED SECTOR
                        *
FDE7 37         GETSCT: PSHB
FDE8 F60263             LDAB    SCTWNT          *LOAD REQUIRED SECTOR
FDEB C10F               CMPB    I,15
FDED 2218               BHI     NOSECT          *IF SECTOR NUMBER > 15, HAVE ERROR
FDEF 36                 PSHA
FDF0 5A                 DECB                    *DECREMENT ACCB TO POINT TO PREVIOUS SECTOR
FDF1 C40F               ANDB    I,0FH
FDF3 8D09               BSR     SCTGET          *SEEK REQUIRED SECTOR - 1
FDF5 F60263             LDAB    SCTWNT          *LOAD REQUIRED SECTOR
FDF8 8D04               BSR     SCTGET          *SEEK REQUIRED SECTOR
FDFA 32                 PULA
FDFB 33                 PULB
```

```
FDFC 0C                 CLC
FDFD 39                 RTS
FDFE B68000     SCTGET: LDAA    STATUS          ;LOAD STATUS BYTE
FE01 840F               ANDA    I,SECTOR        ;GET CURRENT SECTOR
FE03 11                 CBA                     ;COMPARE WITH REQUIRED SECTOR
FE04 26F8               BNE     SCTGET          ;IF NOT SAME, LOOP UNTIL THEY ARE
FE06 39                 RTS                     ;IF SAME, EXIT
FE07 33         NOSECT: PULB
FE08 C607               LDAB    I,SCTERR        ;ERROR - SECTOR NUMBER INVALID
FE0A 0D                 SEC
FE0B 39                 RTS
                        ;
                        ;
                        ;CHECK IF BLOCK WRITTEN CORRECTLY
                        ;
FE0C 7D0270     CHKWRT: TST     CHKSTS          ;SEE IF IN VERIFY OR NON-VERIFY MODE
FE0F 2602               BNE     CWRITE          ;BRANCH FOR VERIFY MODE
FE11 0C                 CLC
FE12 39                 RTS
FE13 C603       CWRITE: LDAB    I,3
FE15 F7026D             STAB    RDCNT           ;MOVE NUMBER OF READ ATTEMPTS TO READ COUNT
FE18 2022               BRA     READLP          ;GO TO READ LOOP
                        ;
                        ;
                        ;READ A 256 BYTE BLOCK FROM DISC
                        ;
FE1A 8D16       READ:   BSR     READ3           ;TRY 3 READS
FE1C 2411               BCC     READOK          ;IF OK, EXIT
FE1E BDFC88             JSR     STEP            ;IF ERROR, STEP HEAD ONE TRACK
FE21 8D0F               BSR     READ3           ;TRY 3 READS
FE23 240A               BCC     READOK          ;IF OK, EXIT
FE25 BDFC18             JSR     TRKZER          ;IF ERROR, SEEK TRACK ZERO
FE28 2504               BCS     READER
FE2A 8D06               BSR     READ3           ;TRY 3 READS
FE2C 2401               BCC     READOK          ;IF OK, EXIT
FE2E 39         READER: RTS                     ;IF ERROR, EXIT WITH ERROR CODE IN ACCB
FE2F 5F         READOK: CLRB
FE30 0C                 CLC
FE31 39                 RTS
                        ;
                        ;
                        ;TRY 3 READS FOR ERROR-FREE TRANSFER
                        ;
FE32 C603       READ3:  LDAB    I,3
FE34 F7026D             STAB    RDCNT           ;MOVE NUMBER OF READ ATTEMPTS TO READ COUNT
FE37 BDFC58             JSR     DSKSEL          ;SELECT REQUIRED DRIVE
FE3A 250E               BCS     RDEXIT
FE3C BDFCAA     READLP: JSR     GETTRK          ;SEEK REQUIRED TRACK
FE3F 2509               BCS     RDEXIT
FE41 8D08               BSR     READ1           ;TRY 1 READ
FE43 2405               BCC     RDEXIT          ;IF OK, EXIT
FE45 7A026D             DEC     RDCNT           ;IF ERROR, DECREMENT READ COUNT
FE48 26F2               BNE     READLP          ;IF ERROR IN 3 ATTEMPTS, EXIT WITH ERROR IN ACCB
FE4A 39         RDEXIT: RTS
                        ;
```

```
                        ;TRY 1 READ FOR ERROR-FREE TRANSFER
                        ;
FE4B BDFD09     READ1:  JSR     HDLOAD          ;LOAD HEAD
FE4E CE0140             LDX     I,RDBUFF        ;LOAD START OF READ BUFFER INTO IX
FE51 86C2               LDAA    I,SYNSCH        ;LOAD DATA SYNC SEARCH CODE INTO ACCA
FE53 8D92               BSR     GETSCT          ;SEEK REQUIRED SECTOR
FE55 2546               BCS     RD1ERR
FE57 B74010             STAA    SSDAC1          ;OUTPUT DATA SYNC CODE TO SERIAL INTERFACE
FE5A C601       RDLOOP: LDAB    I,RDA           ;TEST RECEIVE DATA AVAILABLE BIT
FE5C F54010             BITB    SSDAST
FE5F 27F9               BEQ     RDLOOP
FE61 B64011             LDAA    SSDAIP          ;IF DATA AVAILABLE, LOAD INTO ACCA
FE64 A700               STAA    0,X             ;STORE IN BUFFER
FE66 08                 INCX                    ;INCREMENT BUFFER POINTER
FE67 8C025C             CPX     I,RBUFND+2      ;TEST FOR END OF READ BUFFER
FE6A 26EE               BNE     RDLOOP
FE6C 86CB               LDAA    I,RESET         ;IF END OF BLOCK
FE6E B74010             STAA    SSDAC1          ;MOVE RESET CODE TO SERIAL INTERFACE
FE71 CE0140             LDX     I,RDBUFF        ;GET POINTER TO READ BUFFER
FE74 BDFDCB             JSR     CKSUM           ;CALCULATE CHECKSUM
FE77 F10259             CMPB    RCKSUM
FE7A 2617               BNE     CKERR
FE7C B1025A             CMPA    RCKSUM+1        ;COMPARE WITH CHECKSUM READ FROM DISK
FE7F 2612               BNE     CKERR
FE81 B60262             LDAA    TRKWNT
FE84 B1014B             CMPA    TRAKIN          ;COMPARE REQUIRED TRACK WITH READ TRACK
FE87 260E               BNE     TRAKER
FE89 B60263             LDAA    SCTWNT
FE8C B1014C             CMPA    SECTIN          ;COMPARE REQUIRED SECTOR WITH READ SECTOR
FE8F 260A               BNE     SECTER
FE91 0C                 CLC
FE92 39                 RTS
FE93 C608       CKERR:  LDAB    I,CKSMER        ;ERROR - CHECKSUM ERROR
FE95 2006               BRA     RD1ERR
FE97 C609       TRAKER: LDAB    I,TRKER         ;ERROR - READ TRACK <> REQUIRED TRACK
FE99 2002               BRA     RD1ERR
FE9B C610       SECTER: LDAB    I,SECER         ;ERROR - READ SECTOR <> REQUIRED SECTOR
FE9D 0D         RD1ERR: SEC
FE9E 39                 RTS
                        ;
                        ;
                        ;SET UP THE SSDA
                        ;
FE9F 860B       SETUP:  LDAA    I,SELC2         ;MOVE SETUP CODES TO SERIAL INTERFACE
FEA1 B74010             STAA    SSDAC1
FEA4 861C               LDAA    I,SETC2
FEA6 B74011             STAA    SSDACR
FEA9 864B               LDAA    I,SELC3
FEAB B74010             STAA    SSDAC1
FEAE 860E               LDAA    I,SETC3
FEB0 B74011             STAA    SSDACR
FEB3 868B               LDAA    I,SELSYN
FEB5 B74010             STAA    SSDAC1
FEB8 8681               LDAA    I,SYNC
FEBA B74011             STAA    SSDACR
FEBD 86CB               LDAA    I,RESET
```

```
FEBF B74010             STAA    SSDAC1
FEC2 39                 RTS
                        }
                        }
                        #FORMAT A NEW DISK
                        }
FEC3 864C       FORMAT: LDAA    I,76
FEC5 B70262             STAA    TRKWNT          #MOVE HIGHEST TRACK NUMBER TO REQUIRED TRACK
FEC8 4F                 CLRA
FEC9 CE0000             LDX     I,WRTBUF        #GET POINTER TO START OF WRITE BUFFER
FECC A700       FLOOP:  STAA    0,X             #CLEAR MEMORY LOCATION
FECE 08                 INCX                    #INCREMENT POINTER
FECF 8C013C             CPX     I,WBUFND+17
FED2 26F8               BNE     FLOOP           #LOOP UNTIL WRITE BUFFER IS ALL ZEROS
FED4 BDFC58             JSR     DSKSEL          #SELECT REQUIRED DRIVE
FED7 7F0263             CLR     SCTWNT          #SET REQUIRED SECTOR = ZERO
FEDA BDFC18             JSR     TRKZER          #SEEK TRACK ZERO TO CHECK HEAD POSITION
FEDD B60262     TRKLOP: LDAA    TRKWNT
FEE0 971C               STAA    TRKNUM          #SETUP TRACK NUMBER IN BUFFER
FEE2 7F001D             CLR     SCTNUM          #CLEAR SECTOR NUMBER
FEE5 961C               LDAA    TRKNUM
FEE7 B7012B             STAA    WCKSUM+1        #CALCULATE CHECKSUM
FEEA BDFD84             JSR     WSETUP          #SETUP FOR WRITE TRANSFER
FEED BDFDAF     SCTLOP: JSR     WRTLOP          #WRITE SECTOR TO DISK
FEF0 D61D               LDAB    SCTNUM
FEF2 5C                 INCB                    #INCREMENT SECTOR NUMBER
FEF3 C40F               ANDB    I,SECTOR
FEF5 270F               BEQ     NXTTRK          #TEST FOR LAST SECTOR ON CURRENT TRACK
FEF7 D71D               STAB    SCTNUM          #IF NOT, SAVE NEW SECTOR NUMBER
FEF9 7C012B             INC     WCKSUM+1        #INCREMENT CHECKSUM TO ACCOUNT FOR NEW SECTOR
FEFC BDFDFE             JSR     SCTGET          #GET START OF NEXT SECTOR
FEFF CE0000             LDX     I,WRTBUF        #GET START OF WRITE BUFFER
FF02 08                 INCX                    #SHORT DELAY
FF03 09                 DEX                     #SHORT DELAY
FF04 20E7               BRA     SCTLOP          #WRITE NEXT SECTOR
FF06 BDFD76     NXTTRK: JSR     WCLOSE          #IF LAST SECTOR FOR CURRENT TRACK, END WRITE
FF09 7A0262             DEC     TRKWNT          #DECREMENT TRACK NUMBER
FF0C 2ACF               BPL     TRKLOP
FF0E 7F0262             CLR     TRKWNT
FF11 7F0263             CLR     SCTWNT
FF14 5F                 CLRB
FF15 39                 RTS
                        }
                        }
                        #DISK COMMAND TABLE - LIST OF COMMAND ROUTINE ENTRY POINTS
                        }
FF16 FE1A       CTABLE: FDB     READ            #OPCODE = 1
FF18 FD2C               FDB     WRITE           #OPCODE = 2
FF1A FD25               FDB     VWRITE          #OPCODE = 3
FF1C FCAA               FDB     GETTRK          #OPCODE = 4
FF1E FD09               FDB     HDLOAD          #OPCODE = 5
FF20 FCEE               FDB     DISSEL          #OPCODE = 6
FF22 FC00               FDB     INIT            #OPCODE = 7
FF24 FEC3       ENDTBL: FDB     FORMAT          #OPCODE = 8
                        }
                        }
```

```
                        ;MAIN CONTROL LOOP - ENTRY POINT IS AT 'RENTRY'
                        ;
FF26 B60260             DCONTL: LDAA    OPCODE          ;LOAD OPCODE
FF29 2725                       BEQ     NOTCMD          ;SEE IF VALID
FF2B 8108                       CMPA    I,8
FF2D 2221                       BHI     NOTCMD
FF2F CEFF14                     LDX     I,CTABLE-2      ;LOAD COMMAND TABLE POINTER
FF32 8CFF24             DCLOOP: CPX     I,ENDTBL
FF35 2719                       BEQ     NOTCMD
FF37 08                         INCX                    ;INCREMENT POINTER
FF38 08                         INCX                    ;INCREMENT POINTER
FF39 4A                         DECA                    ;DECREMENT OPCODE
FF3A 26F6                       BNE     DCLOOP          ;LOOP UNTIL OPCODE = ZERO
FF3C 5F                         CLRB
FF3D EE00                       LDX     0,X             ;LOAD COMMAND ROUTINE ENTRY POINT
FF3F AD00                       JSR     0,X             ;CALL COMMAND ROUTINE
                        ;
FF41 F70264             RENTRY: STAB,   ERCODE          ;ON RETURN, STORE ERROR CODE
FF44 7F0260             ENDCMD: CLR     OPCODE          ;CLEAR OPCODE LOCATION
FF47 0E                         CLI                     ;CLEAR INTERRUPT FLAG (TO ENABLE INTERRUPTS)
FF48 3E                         WAI                     ;WAIT FOR INTERRUPT (RUN COMMAND)
FF49 01                         NOOP
FF4A 01                         NOOP
FF4B 01                         NOOP
FF4C 01                         NOOP
FF4D 0F                         SEI                     ;WHEN RUN COMMAND RECEIVED, DISABLE INTERRUPTS
FF4E 20D6                       BRA     DCONTL
FF50 C611               NOTCMD: LDAB    I,NOCMND        ;ERROR - INVALID COMMAND NUMBER
FF52 20ED                       BRA     RENTRY
                        ;
                        ;
                        ;RESTART ROUTINE
                        ;
FF54 8E03F7             RESTRT: LDS     I,STKPTR        ;SETUP STACK POINTER
FF57 BDFE9F                     JSR     SETUP           ;INITIALISE THE SSDA
FF5A 8604                       LDAA    I,4             ;INITIALISE ALL FOUR DRIVES
FF5C B70261                     STAA    DRVWNT
FF5F BDFC00             INITLP: JSR     INIT
FF62 7A0261                     DEC     DRVWNT
FF65 2AF8                       BPL     INITLP
FF67 7C0261                     INC     DRVWNT
                        ;
FF6A CE0008                     LDX     I,LOADTS        ;GET LOADER TRACK-SECTOR
FF6D FF0262                     STX     TRKWNT
FF70 BDFE1A                     JSR     READ            ;READ LOADER INTO READ BUFFER
FF73 20CC                       BRA     RENTRY          ;BRANCH TO COMMAND ROUTINE
                        ;
                        ;
                        ;IRQ SERVICE ROUTINE
                        ;(STARTS COMMAND EXECUTION)
                        ;
FF75 3B                 INTREQ: RTI                     ;ON IRQ, CONTINUE COMMAND LOOP
                        ;
                        ;
                        ;NMI SERVICE ROUTINE
                        ;(GENERATED BY DISK I/O FAIL CONDITION)
```

```
                        ;
FF76 C612               NMIREQ: LDAB    I,DSKIO         ;ERROR - DISK I/O ERROR
FF78 8E03F7                     LDS     I,STKPTR        ;RESTORE STACK POINTER
FF7B 20C4                       BRA     RENTRY
                        ;
                        ;
                        ;INTERRUPT AND RESTART VECTORS
                        ;
FFF8                            ORG     0FFF8H
                        ;
FFF8 FF75               IRQVEC: FDB     INTREQ
FFFA FF54               SWIVEC: FDB     RESTRT
FFFC FF76               NMIVEC: FDB     NMIREQ
FFFE FF54               MRESET: FDB     RESTRT
                        ;
                        ;
                                END

ASSEMBLY COMPLETE,  NO ERRORS
```